

FreeRTOS基础（六）：中断管理

024-06-02 17:04:48 修改

在嵌入式系统开发中，中断管理是一个至关重要的概念。它允许我们的系统能够及时响应外部事件，而不需要通过轮询的方式来浪费宝贵的处理器资源。FreeRTOS 作为一款广泛应用的系统，它提供了灵活且高效的中断管理机制，可以帮助我们更好地处理这些外部事件。

一、什么是中断？

- 1.1 中断的概念
- 1.2 中断执行机制
- 1.3 Cortex-M4的中断资源
- 1.4 两个重要的内核中断
 - 1.4.1 PendSV中断
 - 1.4.2 SysTick

二、中断优先级分组设置(熟悉)

- 2.1 外部中断
- 2.2 内部中断
- 2.3 中断优先级分组设置
 - 2.3.1 优先级分组概念
 - 2.3.1 FreeRTOS优先级分组配置
 - 2.3.2 特点

三、中断相关寄存器(熟悉)

- 3.1 系统中断优先级配置寄存器
- 3.2 FreeRTOS如何配置PendSV和SysTick中断优先级？
- 3.2 中断屏蔽寄存器
 - 3.2.1 FreeRTOS关中断和开中断的函数API

四、FreeRTOS中断管理实验(掌握)编辑

- 6.1 定时器6和定时器7的初始化配置
- 6.2 创建任务及实现任务函数
- 6.3 主函数调用入口函数，操作系统开始进行任务的切换和调度
- 6.4 实验结果

内容来源：csdn.net

作者昵称：未来可期，静待花开~

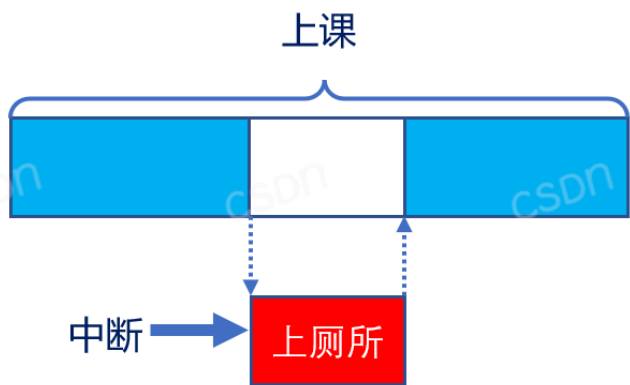
原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页：https://blog.csdn.net/weixin_47040031

什么是中断?

中断的概念

让CPU打断正常运行的程序，转而去处理紧急的事件（中断服务函数ISR），当中断事件处理完毕后，处理器可以恢复到中断前的状态，继续执行之前的程序，就叫中断。



CSDN @未来可期，静待花开~

中断执行机制

中断执行机制，可简单概括为三步：

- 1，中断请求 外设产生中断请求（GPIO外部中断、定时器中断等）
- 2，响应中断 CPU停止执行当前程序，转而去执行中断处理程序（ISR）
- 3，退出中断 执行完毕，返回被打断的程序处，继续往下执行

CSDN @未来可期，静待花开~

Cortex-M4的中断资源

Cortex-M4内核支持256个中断，包含16个系统（内核）中断和240个外部中断。但是芯片厂商一般不会对内核的这些资源全部用完。经过裁剪得到自己厂家的芯片。内核中断如下所示：

内容来源：[csdn.net](https://blog.csdn.net/weixin_47040031/article/details/139380669)
作者昵称：未来可期，静待花开~
原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669
作者主页：https://blog.csdn.net/weixin_47040031

编号	类型	优先级	简介
0	N/A	N/A	没有异常在运行
1	复位	-3 (最高)	复位
2	NMI	-2	不可屏蔽中断 (来自外部 NMI 输入脚)
3	硬(hard)fault	-1	所有被除能的 fault, 都将“上访” (escalation)成硬 fault。只要 FAULTMASK 没有置位, 硬 fault 服务例程就被强制执行。Fault 被除能的原因包括被禁用, 或者 FAULTMASK 被置位。
4	MemManage fault	可编程	存储器管理 fault, MPU 访问犯规以及访问非法位置均可引发。企图在“非执行区”取指也会引发此 fault
5	总线 fault	可编程	从总线系统收到了错误响应, 原因可以是预取流产 (Abort) 或数据流产, 或者企图访问协处理器
6	用法(usage) Fault	可编程	由于程序错误导致的异常。通常是使用了一条无效指令, 或者是非法的状态转换, 例如尝试切换到 ARM 状态
7-10	保留	N/A	N/A
11	SVCall	可编程	执行系统服务调用指令 (SVC) 引发的异常
12	调试监视器	可编程	调试监视器 (断点, 数据观察点, 或者是外部调试请求
13	保留	N/A	N/A
14	PendSV	可编程	为系统设备而设的“可悬挂请求” (pendable request)
15	SysTick	可编程	系统滴答定时器 (也就是周期性溢出的时基定时器。——译注)

裁剪, **STM32F407具有82个可屏蔽中断通道。10个系统中断**, 如下图所示:

位置	优先级	优先级类型	名称	说明	地址
	-	-	-	保留	0x0000 0000
-3	固定	Reset	复位		0x0000 0004
-2	固定	NMI	不可屏蔽中断。RCC 时钟安全系统 (CSS) 连接到 NMI 向量。		0x0000 0008
-1	固定	HardFault	所有类型的错误		0x0000 000C
0	可设置	MemManage	存储器管理		0x0000 0010
1	可设置	BusFault	预取指失败，存储器访问失败		0x0000 0014
2	可设置	UsageFault	未定义的指令或非法状态		0x0000 0018
-	-	-	保留		0x0000 001C - 0x0000 002B
3	可设置	SVCall	通过 SWI 指令调用的系统服务		0x0000 002C
4	可设置	Debug Monitor	调试监控器		0x0000 0030
-	-	-	保留		0x0000 0034
5	可设置	PendSV	可挂起的系统服务		0x0000 0038
6	可设置	SysTick	系统嘀嗒定时器		0x0000 003C
0	7	可设置	WWDG	窗口看门狗中断	0x0000 0040
1	8	可设置	PVD	连接到 EXTI 线的可编程电压检测 (PVD) 中断	0x0000 0044
2	9	可设置	TAMP_STAMP	连接到 EXTI 线的入侵和时间戳中断	0x0000 0048
3	10	可设置	RTC_WKUP	连接到 EXTI 线的 RTC 唤醒中断	0x0000 004C
4	11	可设置	FLASH	Flash 全局中断	0x0000 0050
5	12	可设置	RCC	RCC 全局中断	0x0000 0054
6	13	可设置	EXTI0	EXTI 线 0 中断	0x0000 0058
7	14	可设置	EXTI1	EXTI 线 1 中断	0x0000 005C
8	15	可设置	EXTI2	EXTI 线 2 中断	0x0000 0060
9	16	可设置	EXTI3	EXTI 线 3 中断	0x0000 0064
10	17	可设置	EXTI4	EXTI 线 4 中断	0x0000 0068
11	18	可设置	DMA1_Stream0	DMA1 流 0 全局中断	0x0000 006C
12	19	可设置	DMA1_Stream1	DMA1 流 1 全局中断	0x0000 0070
13	20	可设置	DMA1_Stream2	DMA1 流 2 全局中断	0x0000 0074
14	21	可设置	DMA1_Stream3	DMA1 流 3 全局中断	0x0000 0078
15	22	可设置	DMA1_Stream4	DMA1 流 4 全局中断	0x0000 007C
16	23	可设置	DMA1_Stream5	DMA1 流 5 全局中断	0x0000 0080
17	24	可设置	DMA1_Stream6	DMA1 流 6 全局中断	0x0000 0084
18	25	可设置	ADC	ADC1、ADC2 和 ADC3 全局中断	0x0000 0088

内核中断

两个重要的内核中断

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页: https://blog.csdn.net/weixin_47040031

1 PendSV中断

在FreeRTOS中，PendSV（Pended [System](#) Service Call）是一种专门的中断，**用于任务切换**。它是Cortex-M系列处理器（如ARM Cortex-M3、M4、M7等）提供的一种系统级中断，其优先级最低，以确保它只在所有其他中断处理完毕后才执行。

endSV的作用：

PendSV的主要作用是实现上下文切换，即从一个任务切换到另一个任务。这是FreeRTOS实现多任务调度的核心机制之一。具体来说，当需要进行任务切换时，FreeRTOS会设置endSV中断挂起（pend），以便在下一个合适的时间点执行任务切换。

endSV的优点

- 1. **优先级最低**：由于PendSV中断的优先级最低，确保了它不会打断任何其他正在处理的中断，从而保证了系统的稳定性和中断处理的完整性。
- 2. **简化任务切换**：使用PendSV中断进行任务切换简化了操作系统内核的设计，因为任务切换逻辑被集中在一个统一的中断服务例程中。
- 3. **效率高**：PendSV中断通过硬件机制实现，效率高且开销小，适用于实时操作系统的需求。

2 SysTick

在FreeRTOS中，SysTick（系统[定时器](#)）是一种用于实现系统节拍（tick）的定时器。它是ARM Cortex-M系列处理器中的一个内置定时器，用于**提供周期性中断**，从而驱动[操作系统](#)的节拍。这种**周期性中断对操作系统的调度器至关重要，因为它决定了任务的时间片、延时操作和其他时间相关的功能。**

ysTick的作用

- 1. **生成系统节拍（tick）中断**：SysTick定时器周期性地产生中断信号，每次中断称为一个"tick"。这个节拍频率通常被配置为1毫秒一次，但可以根据需要进行调整。
- 2. **驱动调度器**：每个tick中断都会触发FreeRTOS调度器的运行，检查是否有需要调度的任务。例如，如果一个任务的延时时间结束，调度器会将其状态从阻塞（blocked）变为就绪（ready）。
- 3. **时间管理**：SysTick中断用于实现FreeRTOS中的各种时间管理功能，如任务延时（vTaskDelay）、定时器服务（software timers）、和时间片轮转（time slicing）等。

ysTick的工作流程

- 1. **初始化SysTick**：在系统启动时，FreeRTOS会配置SysTick定时器的频率，使其以设定的周期产生中断。这个配置通常在FreeRTOS的初始化代码中完成。
- 2. **产生中断**：SysTick定时器按照设定的周期（例如，每1毫秒）产生中断信号。
- 3. **中断处理**：每次SysTick中断发生时，处理器会跳转到SysTick中断服务例程（ISR）。在FreeRTOS中，SysTick中断服务例程通常会调用xPortSysTickHandler()函数。
- 4. **调度任务**：在SysTick中断服务例程中，FreeRTOS的调度器会执行任务调度逻辑，包括检查延时任务、管理时间片、更新系统时钟等。如果有需要切换的任务，会通过PendSV中断来进行上下文切换。

SysTick在FreeRTOS中扮演着关键角色，它通过周期性中断驱动操作系统的时钟节拍，使得任务调度、延时管理和时间片轮转等功能得以实现。利用Cortex-M系列处理器内置的SysTick定时器，FreeRTOS能够高效且准确地管理系统时间，确保实时操作系统的各项功能正常运行。

内容来源: [csdn.net](https://blog.csdn.net/weixin_47040031/article/details/139380669)
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669
作者主页: https://blog.csdn.net/weixin_47040031

中断优先级分组设置(熟悉)

外部中断

Cortex-M处理器有多个用于管理中断的可编程寄存器。这些寄存器大多数在NVIC（嵌套向量中断控制器，每个中断通道都拥有16个可编程的优先等级，可对优先级进行分组。）和系统控制块（SCB）中。

```
typedef struct
{
    __IOM uint32_t ISER[8U];
    uint32_t RESERVED0[24U];
    __IOM uint32_t ICER[8U];
    uint32_t RESERVED1[24U];
    __IOM uint32_t ISPR[8U];
    uint32_t RESERVED2[24U];
    __IOM uint32_t ICPR[8U];
    uint32_t RESERVED3[24U];
    __IOM uint32_t IABR[8U];
    uint32_t RESERVED4[56U];
    __IOM uint8_t IP[240U];
    uint32_t RESERVED5[644U];
    __IOM uint32_t STIR;
} NVIC_Type;

typedef struct
{
    __IOM uint32_t CPUID;
    __IOM uint32_t ICSR;
    __IOM uint32_t VTOR;
    __IOM uint32_t AIRCR;
    __IOM uint32_t SCR;
    __IOM uint32_t CCR;
    __IOM uint8_t SHP[12U];
    __IOM uint32_t SHCSR;
    __IOM uint32_t CFSR;
    __IOM uint32_t HFSR;
    __IOM uint32_t DFSR;
    __IOM uint32_t MMFAR;
    __IOM uint32_t BFAR;
    __IOM uint32_t AFSR;
    __IOM uint32_t PFR[2U];
    __IOM uint32_t DFR;
    __IOM uint32_t ADR;
    __IOM uint32_t MMFR[4U];
    __IOM uint32_t ISAR[5U];
    uint32_t RESERVED0[5U];
    __IOM uint32_t CPACR;
} SCB_Type;
```

CSDN @未来可期，静待花开~

IP [240]（Interrupt Priority Registers）：**中断优先级控制寄存器组**。由240个8bit的寄存器组成，每个可屏蔽中断占用8bit。由于我们只有82个可屏蔽中断通道，因此我们只用IP[81] - IP[0]这82个，并且，每个中断通道占用的8bit并没有全部使用，而是只用了高4位。我们知道STM32的中断资源由NVIC来统一管理，而每个中断的优先级是由优先级寄存器IP控制的，并且每一个8bit对应一个中断。但是，只占用了高四位。然后我们对这4位又进行了划分，分为高n位的抢占优先级和低4-n位的响应优先级。NVIC里面的**中断优先级控制寄存器组IP可以对所有的外部中断优先级设置。**

内部中断

在ARM Cortex-M处理器中，系统处理优先级（System Handler Priority, SHP）寄存器用于设置系统异常/内核中断（如硬故障、SVCall、SysTick等）的优先级。SHP寄存器是ARM Cortex-M处理器内核的一部分，不是FreeRTOS特有的。**它们用于配置特定系统异常/内部中断的优先级。**

中断优先级分组设置

1.1 优先级分组概念

学过裸机开发，我们知道 ARM Cortex-M 使用了 8 位宽的寄存器来配置中断的优先等级（0-255），这个寄存器就是中断优先级配置寄存器IP，但STM32，只用了中断优先级配置寄存器的低8位，所以STM32**提供了最大16级的中断优先等级。注意：中断优先级数值越小，优先级越高。**

内容来源: csdn.net
作者昵称: 未来可期，静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669
作者主页: https://blog.csdn.net/weixin_47040031

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
用于表达优先级				没有实现，读回零			

CSDN @未来可期，静待花开~

32 的中断优先级可以分为抢占优先级和响应优先级：

1. **抢占优先级**：抢占优先级高的中断可以打断正在执行但抢占优先级低的中断。
2. **响应优先级**：当同时发生具有相同抢占优先级的两个中断时，子优先级数值小的优先执行，如果两个相同抢占优先级的中断不是同时发生，那么是不会发生抢占的，等先发生中断的执行完，才会去执行后来的中断。

1 FreeRTOS优先级分组配置

一共有 5 种分配方式，对应着中断优先级分组的 5 个组：

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页：https://blog.csdn.net/weixin_47040031

优先级分组	抢占优先级	子优先级	优先级配置寄存器高 4 位
NVIC_PriorityGroup_0	0 级抢占优先级	0-15 级子优先级	0bit 用于抢占优先级 4bit 用于子优先级
NVIC_PriorityGroup_1	0-1 级抢占优先级	0-7 级子优先级	1bit 用于抢占优先级 3bit 用于子优先级
NVIC_PriorityGroup_2	0-3 级抢占优先级	0-3 级子优先级	2bit 用于抢占优先级 2bit 用于子优先级
NVIC_PriorityGroup_3	0-7 级抢占优先级	0-1 级子优先级	3bit 用于抢占优先级 1bit 用于子优先级
NVIC_PriorityGroup_4	0-15 级抢占优先级	0 级子优先级	4bit 用于抢占优先级 0bit 用于子优先级

FreeRTOS采用的就是这种分组，这样就可以打断任务

通过调用函数 `NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4);`即可完成设置

CSDN @未来可期，静待花

2 特点

低于configMAX_SYSCALL_INTERRUPT_PRIORITY优先级（也就是处于5-15优先级）的中断里才允许调用FreeRTOS 的API函数；FreeRTOS能管理的最高的优先级。

建议将所有优先级位指定为抢占优先级位，方便FreeRTOS管理。

cpp

AI写代码

复制

运行

1

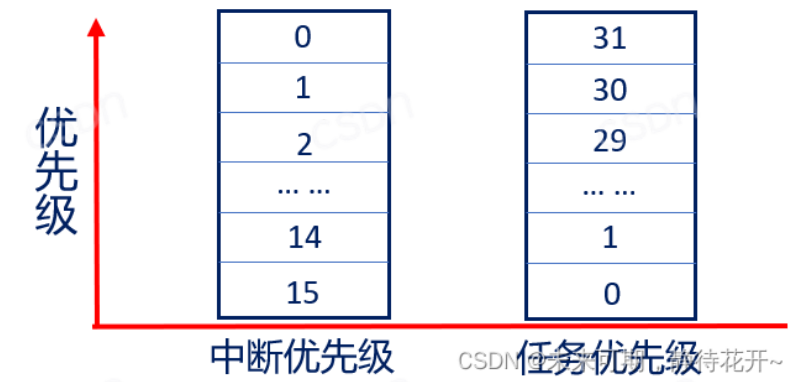
(调用函数)

2

NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4);

中断优先级数值越小越优先，任务优先级数值越大越优先。

内容来源: csdn.net
 作者昵称: 未来可期，静待花开~
 原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669
 作者主页: https://blog.csdn.net/weixin_47040031



中断相关寄存器(熟悉)

系统中断优先级配置寄存器

系统控制块SCB结构体存在一个系统处理优先级（System Handler Priority, SHP）寄存器用于设置系统异常/内核中断（如硬故障、SVCall、SysTick等）的优先级。SHP寄存器是ARM Cor里器内核的一部分，不是FreeRTOS特有的。**它们用于配置特定系统异常/内部中断的优先级。**

内容来源: csdn.net
作者昵称: 未来可期，静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669
作者主页: https://blog.csdn.net/weixin_47040031

三个系统中断优先级配置寄存器，分别为 SHPR1、SHPR2、SHPR3

SHPR1寄存器地址：0xE000ED18

SHPR2寄存器地址：0xE000ED1C

SHPR3寄存器地址：0xE000ED20

表8.3B 系统异常优先级寄存器阵列

0xE000_ED18 - 0xE000_ED23

地址	名称	类型	复位值	描述
0xE000_ED18	PRI_4			存储器管理 fault 的优先级
0xE000_ED19	PRI_5			总线 fault 的优先级
0xE000_ED1A	PRI_6			用法 fault 的优先级
0xE000_ED1B	-	-	-	-
0xE000_ED1C	-	-	-	-
0xE000_ED1D	-	-	-	-
0xE000_ED1E	-	-	-	-
0xE000_ED1F	PRI_11			SVC 优先级
0xE000_ED20	PRI_12			调试监视器的优先级
0xE000_ED21	-	-	-	-
0xE000_ED22	PRI_14			PendSV 的优先级 (进行任务的切换和调度)
0xE000_ED23	PRI_15			SysTick 的优先级 (系统节拍定时器)

CSDN @未来可期，静待花

四个8位的寄存器组成一个32位的寄存器，所以，pendsv和systick的控制寄存器为：0xE000ED20 的次高8位和高8位。

FreeRTOS如何配置PendSV和Systick中断优先级？

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页：https://blog.csdn.net/weixin_47040031

```

/* Make PendSV and SysTick the lowest priority interrupts. */
portNVIC_SHPR3_REG |= portNVIC_PENDSV_PRI;
portNVIC_SHPR3_REG |= portNVIC_SYSTICK_PRI;

```

```

Constants required to manipulate the core registers first... */
#define portNVIC_SYSTICK_CTRL_REG (*((volatile uint32_t *) 0x000e010))
#define portNVIC_SYSTICK_LOAD_REG (*((volatile uint32_t *) 0x000e014))
#define portNVIC_SYSTICK_CURRENT_VALUE_REG (*((volatile uint32_t *) 0x000e018))
#define portNVIC_SHPR3_REG (*((volatile uint32_t *) 0x000e020))

```

```

#define portNVIC_PENDSV_PRI ((uint32_t) configKERNEL_INTERRUPT_PRIORITY << 16UL)
#define portNVIC_SYSTICK_PRI ((uint32_t) configKERNEL_INTERRUPT_PRIORITY << 24UL)

```

```

/* 中断嵌套行为配置 */
#ifndef __NVIC_PRIO_BITS
#define configPRIO_BITS __NVIC_PRIO_BITS
#else
#define configPRIO_BITS 4
#endif

#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY 15 /* 中断最低优先级 */
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5 /* FreeRTOS可管理的最高中断优先级 4 */
#define configKERNEL_INTERRUPT_PRIORITY (configLIBRARY_LOWEST_INTERRUPT_PRIORITY < (8 - configPRIO_BITS))
#define configMAX_SYSCALL_INTERRUPT_PRIORITY (configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY < (8 - configPRIO_BITS))
#define configMAX_API_CALL_INTERRUPT_PRIORITY configMAX_SYSCALL_INTERRUPT_PRIORITY

```

CSDN@未来可期_静待花开~

1. **#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY**：此宏用来设置Free RTOS系 统可管理的最大优先级。高于此宏设置的优先级不受Free RTOS的管理。
2. **#define configKERNEL_INTERRUPT_PRIORITY**用来设置内核中断优先级：PendSV 和 SYSTICK的中断优先级（左移4位是因为：中断优先级IP只有高4位有效）

通过上述配置，将PendSV和SysTick设置中断优先级中的最低优先级，设置最低：保证系统任务切换不会阻塞系统其他中断的响应，为什么？中断是非常紧急的事情，也就是说中断可以打断任务，而任务不可以打断中断，PendSV就是用来处理任务切换的内核中断，因此把PendSV任务切换中断的优先级配置为最低优先级，这样当中断事件发生的时候，它会打断PendSV，也就一旦发生，就不会在切换任务执行，但是中断发生的时候，PendSV是不会打断中断的，因为它的优先级最低。

中断屏蔽寄存器

所谓FreeRTOS的中断管理，就是利用的BASEPRI这个寄存器，来屏蔽优先级低于某一个阈值的中断，当设置为0的时候，就是打开所有中断，所有中断都可以响应。

三个中断屏蔽寄存器，分别为 PRIMASK、FAULTMASK 和BASEPRI

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页：https://blog.csdn.net/weixin_47040031

名字	功能描述
PRIMASK	这是个只有 1 个位的寄存器。当它置 1 时，就关掉所有可屏蔽的异常，只剩下 NMI 和硬 fault 可以响应。它的缺省值是 0，表示没有关中断。
FAULTMASK	这是个只有 1 个位的寄存器。当它置 1 时，只有 NMI 才能响应，所有其它的异常，包括中断和 fault，通通闭嘴。它的缺省值也是 0，表示没有关异常。
BASEPRI	这个寄存器最多有 9 位（由表达优先级的位数决定）。它定义了被屏蔽优先级的阈值。当它被设成某个值后，所有优先级号大于等于此值的中断都被关（优先级号越大，优先级越低）。但若被设成 0，则不关闭任何中断，0 也是缺省值。

CSDN @未来可期，静待花开~

BASEPRI：屏蔽优先级低于某一个阈值的中断。比如：BASEPRI设置为0x50，代表中断优先级在5~15内的均被屏蔽，0~4的中断优先级正常执行。

断程序示例：

内容来源：csdn.net
 作者昵称：未来可期，静待花开~
 原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669
 作者主页：https://blog.csdn.net/weixin_47040031

```

#define portDISABLE_INTERRUPTS()          vPortRaiseBASEPRI()    关中断函数的宏定义
static portFORCE_INLINE void vPortRaiseBASEPRI( void )
{
    uint32_t ulNewBASEPRI = configMAX_SYSCALL_INTERRUPT_PRIORITY;  FreeRTOS可管理的最高优先级，默认配置为5
    __asm
    {
        msr basepri, ulNewBASEPRI  将这个值写入到basepri寄存器
        dsb
        isb
    }
}

#define configMAX_SYSCALL_INTERRUPT_PRIORITY ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 -
configPRIO_BITS) )

#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5  /* FreeRTOS可管理的最高中断优先级 */

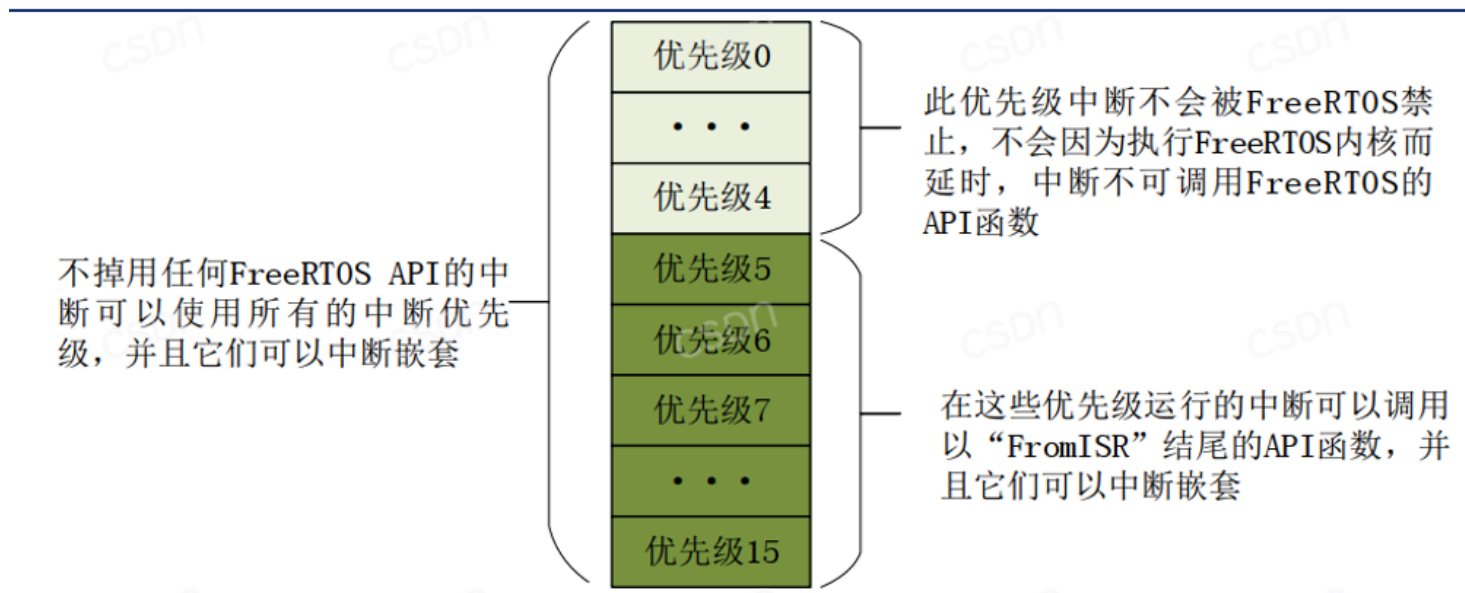
```

CSDN @未来可期，静待花开

可以看到：把configMAX_SYSCALL_INTERRUPT_PRIORITY的值赋给了寄存器“basepri”，此寄存器就是用来屏蔽中断的。优先级的数值大于等于此值的中断将会被屏蔽掉。即中断优先级在5 ~ 15的中断全部被关闭，而中断优先级在0-4的中断不会受影响！

内容来源: csdn.net
 作者昵称: 未来可期，静待花开~
 原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669
 作者主页: https://blog.csdn.net/weixin_47040031

当BASEPRI设置为0x50时:



在中断服务函数中调度FreeRTOS的API函数需注意:

- 1、中断服务函数的优先级需在FreeRTOS所管理的范围内
- 2、在中断服务函数里边需调用FreeRTOS的API函数, 必须使用带“FromISR”后缀的函数

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页: https://blog.csdn.net/weixin_47040031

开中断程序示例:

开中断函数宏定义

```
#define portENABLE_INTERRUPTS()          vPortSetBASEPRI( 0 )  
  
static portFORCE_INLINE void vPortSetBASEPRI( uint32_t ulBASEPRI )  
{  
    __asm  
    {  
        msr basepri, ulBASEPRI  
    }  
}
```

其实就是将basepri寄存器的值设置为0

CSDN @未来可期，静待花开~

1 FreeRTOS关中断和开中断的函数API

关中断函数:

portDISABLE_INTERRUPTS() 或者 **vPortRaiseBASEPRI()**

函数原型: **void vPortRaiseBASEPRI(void)**

开中断函数:

portENABLE_INTERRUPTS() 或者 **vPortSetBASEPRI(0)**

函数原型: **void vPortSetBASEPRI(uint32_t ulBASEPRI)**

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页: https://blog.csdn.net/weixin_47040031

1、实验目的：学会使用FreeRTOS的中断管理！

本实验会使用两个定时器，定时器7优先级为4，定时器6优先级为6，**注意：FreeRTOS系统所管理的优先级范围：5~15**，现象：两个定时器每1s，打印一段字符串，当关中断时，定时器6停止打印，开中断时持续打印。而定时器7不会受到影响，因为它不在FreeRTOS优先级管理范围之内。

2、实验设计：将设计2个任务：start_task、task1

2个任务的功能如下：

start_task

用来创建task1任务

task1

中断测试任务，任务中将调用关中断和开中断函数来体现对中断的管理作用！

CSDN @未来可期，静待花

定时器6和定时器7的初始化配置

```
#include "stm32f4xx.h"           // Device header
#include "mytim.h"
#include "stdio.h"

/***** 配置基本定时器6和基本定时器7 *****/
void Interrupt_Init(void)
{
    //1. 开启TIM时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM6,ENABLE); //开启时钟使能
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM7,ENABLE); //开启时钟使能
```

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页：https://blog.csdn.net/weixin_47040031

```
// 设置中断优先级分组 (FreeRTOS 利用的是抢占式调度) 14 | NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4);
```

```
//2. 配置中断, 配置NVIC
NVIC_InitTypeDef Struct;
Struct.NVIC_IRQChannel = TIM6_DAC_IRQn;
Struct.NVIC_IRQChannelCmd =ENABLE;
Struct.NVIC_IRQChannelPreemptionPriority = 6; //基本定时器6抢占优先级6
Struct.NVIC_IRQChannelSubPriority = 0; //基本定时器6响应优先级0
NVIC_Init(&Struct);
```

```
NVIC_InitTypeDef Struct1;
Struct1.NVIC_IRQChannel = TIM7_IRQn;
Struct1.NVIC_IRQChannelCmd =ENABLE;
Struct1.NVIC_IRQChannelPreemptionPriority = 4; //基本定时器7抢占优先级4
Struct1.NVIC_IRQChannelSubPriority = 0; //基本定时器7响应优先级0
NVIC_Init(&Struct1);
```

```
//3. 配置TIM (时基单元)
TIM_TimeBaseInitTypeDef Struct2;
Struct2.TIM_Period = 9999; //自动重载寄存器ARR: 定时1秒
Struct2.TIM_Prescaler = 8399; //预分频器PSC
TIM_TimeBaseInit(TIM6,&Struct2); //初始化时基单元
```

```
TIM_TimeBaseInitTypeDef Struct3;
Struct3.TIM_Period = 9999; //自动重载寄存器ARR: 定时1秒
Struct3.TIM_Prescaler = 8399; //预分频器PSC
TIM_TimeBaseInit(TIM7,&Struct3); //初始化时基单元
```

```
TIM_ITConfig(TIM6,TIM_IT_Update,ENABLE); //定时中断配置
TIM_Cmd(TIM6,ENABLE); //使能定时器

TIM_ITConfig(TIM7,TIM_IT_Update,ENABLE); //定时中断配置
TIM_Cmd(TIM7,ENABLE); //使能定时器
```

```
}
```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页: https://blog.csdn.net/weixin_47040031

```

57 |
//4. 重写定时器6 中断服务函数
void TIM6_DAC_IRQHandler(void)
{
    if(TIM_GetITStatus(TIM6,TIM_IT_Update) == SET)
    {

        //具体实现
        printf("TIM6优先级为6的定时器正在运行! \n");

        TIM_ClearITPendingBit(TIM6,TIM_IT_Update); //清除中断标志位
    }
}

//4. 重写定时器7 中断服务函数
void TIM7_IRQHandler(void)
{
    if(TIM_GetITStatus(TIM6,TIM_IT_Update) == SET)
    {
        //具体实现
        printf("TIM7优先级为4的定时器正在运行!!! \n");

        TIM_ClearITPendingBit(TIM7,TIM_IT_Update); //清除中断标志位
    }
}

```

创建任务及实现任务函数

```

#include "stm32f4xx.h"           // Device header
#include "stdio.h"
#include "FreeRTOS.h"
#include "task.h"
#include "dynamic.h"
#include "mydelay.h"

/*****START_TASK任务配置*****/

```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页: https://blog.csdn.net/weixin_47040031

```

/***** 包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
#define START_TASK_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
#define START_TASK_PRIO 1 //定义任务优先级，0-31根据任务需求
TaskHandle_t start_task_handler; //定义任务句柄（结构体指针）
void start_task(void* args);

/***** TASK1 任务配置*****/
/***** 包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
#define TASK1_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
#define TASK1_PRIO 2 //定义任务优先级，0-31根据任务需求
TaskHandle_t task1_handler; //定义任务句柄（结构体指针）
void task1(void* args);

/***** 开始任务用来创建一个任务，只创建一次，不能是死循环,创建完这个任务后删除开始任务本身*****/
void start_task(void* args)
{
    taskENTER_CRITICAL(); /*进入临界区*/

    xTaskCreate( (TaskFunction_t) task1,
                (char *) "task1",
                ( configSTACK_DEPTH_TYPE ) TASK1_STACK_SIZE,
                (void *) NULL,
                (UBaseType_t) TASK1_PRIO ,
                (TaskHandle_t *) &task1_handler );

    vTaskDelete(NULL); //删除开始任务自身，传参NULL

    taskEXIT_CRITICAL(); /*退出临界区*/

    // 临界区内不会进行任务的调度切换，出了临界区才会进行任务调度，抢占式
}

```

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页：https://blog.csdn.net/weixin_47040031

```

/***** 任务1的任务函数，无返回值且是死循环*****/

```

```

/** 任务1：实现LED0每500ms翻转一次*****/

```

```

void task1(void* args)

```

```

{

```

```

    uint8_t task1_num=0;

```

```

    while(1)

```

```

    {

```

```

        if(++task1_num==5)

```

```

        {

```

```

            task1_num=0;

```

```

            printf("关中断! \n");

```

```

            portDISABLE_INTERRUPTS(); //将BASEPRI寄存器设置为5

```

```

            My_Delay_s(5); // 这里不能调用vTaskDelay(5000); 因为：这个函数内部调用的是临界区保护，将BASEPRI寄存器设置为0，也就是打开中断，关了又开无法看到实验效果

```

```

            printf("开中断!!! \n");

```

```

            portENABLE_INTERRUPTS(); //将BASEPRI寄存器设置为0，打开中断

```

```

        }

```

```

        vTaskDelay(1000); //FreeRTOS自带的延时函数，延时1秒

```

```

    }

```

```

}

```

```

//FreeRTOS入口例程函数，无参数，无返回值

```

```

void freertos_demo(void)

```

```

{

```

```

    xTaskCreate( (TaskFunction_t) start_task,

```

```

                (char *) "start_task",

```

```

                ( configSTACK_DEPTH_TYPE) START_TASK_STACK_SIZE,

```

```

                (void *) NULL,

```

```

                (UBaseType_t) START_TASK_PRIO,

```

```

                (TaskHandle_t *) &start_task_handler );

```

```

    vTaskStartScheduler(); //开启任务调度器

```

```

}

```

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页：https://blog.csdn.net/weixin_47040031

主函数调用入口函数，操作系统开始进行任务的切换和调度

AI写代码 | 复制 | 运行

```
#include "stm32f4xx.h"           // Device header
#include "stdio.h"
#include "myled.h"
#include "mykey.h"
#include "myusart.h"
#include "mytim.h"

#include "FreeRTOS.h"
#include "task.h"
#include "dynamic.h"

extern TaskHandle_t Start_Handle;

int main(void)
{
    // 硬件初始化
    My_UsartInit();
    LED_Init();
    KEY_Init();
    Interrupt_Init();

    // 调用入口函数
    freertos_demo();
}
```

实验结果

内容来源: [csdn.net](https://blog.csdn.net/weixin_47040031)

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669

作者主页: https://blog.csdn.net/weixin_47040031

```
TIM7优先级为4的定时器正在运行!!!  
关中断!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
开中断!!!  
TIM6优先级为6的定时器正在运行!  
TIM6优先级为6的定时器正在运行!  
TIM7优先级为4的定时器正在运行!!!  
TIM7优先级为4的定时器正在运行!!!  
TIM6优先级为6的定时器正在运行!
```

CSDN @未来可期，静待花开~

本实验创建了两个定时器中断，每隔1秒打印字符串，并且一个定时器中断的优先级在FreeRTOS的管理范围之内，另一个不在FreeRTOS的管理范围之内，这样就可以验证中断管理函数的作用！一开始，两个定时器每隔1秒打印相应的字符串，各自运行5次后，关闭中断，那么定时器6中断就会被屏蔽，因此它不会打印，但定时器7不会受影响，定时器7打印5次后，再打开中断，这时那么定时器6中断就会被打开，两个定时器每隔1秒打印相应的字符串，依次循环往复运行！

，中断管理就已经讲解完毕！初次学习，循序渐进，一步步掌握即可！以上就是全部内容！请务必掌握，创作不易，欢迎大家点赞加关注评论，您的支持是我前进最大的动力！下期再见！

#嵌入式硬件 #FreeRTOS

内容来源: csdn.net
作者昵称: 未来可期，静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139380669
作者主页: https://blog.csdn.net/weixin_47040031